

D - Quests

Process

The question is saying that there are n quests provided, and there d days to do these quests. We can get coins after finish one quests, and the number of coins we want to achieve(c) is also provided. The question wants us to find the maximum gap for the person to do the same quests but still achieve c coins.

Firstly, it is obvious that if there is a quest that already has a reward larger than c , we can just simply do that in the first day and achieve the goal of having c coins no matter how large the gap can be. So this case will just output 'Infinity'.

Secondly, let's pretend the gap is 0, and we choose to do the largest reward quests for every day (n days in total). This is the maximum coins we can get. If this is still less than c , no matter what we do, there is no way we can have reward as c . This case we can just return 'Impossible'.

Now is the tricky part, in my original solution I was trying to caculate the lagest gap k directly by going though all days.

Challenges and Overcoming

Now is the tricky part, in my original solution I was trying to caculate the lagest gap k directly by going though all days. But I shouldn't do this beacuse it way more easy to check whether a given k can lead to getting reward coins larger than c rather than trying to caculate myself. Additionally, there is no point to have a k larger than n because if k is larger than n , every quests will be done at most once already, and this allows k to be infinity, just like the I perviously discussed about. And clearly, k can not be less than 0, which means k is a value in between 0 and $n - 1$ if the pervious 2 cases is not happenning.

Therefore, instead of trying to use

```
int k = 0;
int current_coin_gain = 0;
for (int j = 0; j < d; j++) {
    ...
}
```

to directly calculate k , we can use Binary Search to search for each value in between 0 and $n - 1$.

To do that we still need a helper function to check if with gap k we can achieve c coins as rewards.

```

bool check_k(int k, int d, long long c, vector<long long>& rewards) {
    long long coins = 0;
    for (int i = 0; i < d; i++) {
        int current_day_in_cycle = i % (k + 1);
        if (current_day_in_cycle < rewards.size()) {
            coins += rewards[current_day_in_cycle];
        }
        if (coins >= c) return true;
    }
    return coins >= c;
}

```

The main logic of this function is to check if the maximum rewards we can gain with gap length k is larger than c . Clearly, in order to compute the maximum reward gain, we need to do as much and high-reward quests as we can.

Firstly, rewards already sorted in decreasing order. And we will keep doing the most valueable quests in a period of k , and in between do those less valueable quests.